«Код Будущего — программирование на Python для специалистов по информационной безопасности»

Модуль 2. Файлы, данные, простая обработка и визуализация.

ТЕМА JSON и словари. Вложенные структуры, анализ API ответов средств защиты

Практическое занятие

Цель:

Научиться работать с API и JSON..

Задачи:

Разберемся, что такое формат JSON и как он связан с Python-словарами

Научиться разбирать вложенные структуры

Изучить приёмы парсинга и преобразования JSON-данных

Рассмотреть реальные примеры из области информационной безопасности.

Методические указания и инструкции по выполнению задания

1. **Вспомните и повторите теорию, необходимую для выполнения практического задания**

JSON — текстовый формат для обмена данными (ключ:значение). Используется в API, логах, конфигах.

Связь с Python: JSON-объект ↔ словарь, JSON-массив ↔ список.

Чтение/запись: json.load() / json.dump().

Вложенные структуры: Рекурсивный обход для извлечения данных (например, IP с высоким CVSS).

Пример в ИБ: Анализ ответов WAF/SIEM (фильтрация угроз, преобразование в CSV).

2. **Разберитесь в том, как выполняется задание**

Пример программы для чтения JSON-файла и вывода его содержимого.

Задание 1

Проанализировать JSON-файла с уязвимостями

- Файл vulnreport.json с вложенной структурой
- Прочитать JSON

- Вывести список IP-адресов

Описание для тренажера:

1. Прочитать JSON-файл vulnreport.json.

2. Ожидается, что в файле находится список объектов, и каждый объект содержит поле "ip".

3. Собрать все IP-адреса, которые встречаются в данных.

4. Исключить повторяющиеся IP-адреса.

5. Отсортировать IP-адреса по возрастанию.

6. Вывести каждый IP-адрес на отдельной строке.

7. Если файл vulnreport.json отсутствует, вывести сообщение: Файл vulnreport.json не найден.

Тренажер - 20401-analysis-of-a-file-with-a-vulnerability
Решение:

```
import json


def list_ips(input_file="vulnreport.json"):
    try:
        with open(input_file, "r", encoding="utf-8") as
f:
            data = json.load(f)
    except FileNotFoundError:
        print(f"Файл {input_file} не найден")
        return

    ips = set()
    # ожидаем структуру: список хостов, каждый с полем
"ip"
    for host in data:
        ip = host.get("ip")
        if ip:
```

```
            ips.add(ip)

    for ip in sorted(ips):
        print(ip)


if __name__ == "__main__":
    list_ips()
```

Задание 2

- Найти все уязвимости с CVSS > 8
- Вывести ID и описание

Описание для тренажера

1. Прочитать JSON-файл vulnreport.json.

2. Ожидается, что в файле находится список объектов, каждый объект представляет хост и может содержать список уязвимостей в поле "vulns".

3. В каждой уязвимости проверить значение поля "cvss".

4. Отобрать только те уязвимости, у которых CVSS больше 8.

5. Для каждой такой уязвимости получить её "id" и "description".

6. Вывести на экран строки в формате "ID: описание", по одной уязвимости на строку.

7. Если файл vulnreport.json отсутствует, вывести сообщение: Файл vulnreport.json не найден.

Тренажер - 20402-searching-for-vulnerabilities

Решение:

import json

```
def high_cvss(input_file="vulnreport.json"):
    try:
        with open(input_file, "r", encoding="utf-8") as f:
            hosts = json.load(f)
    except FileNotFoundError:
        print(f"Файл {input_file} не найден")
        return

    results = []
    for host in hosts:
        for v in host.get("vulns", []):
            try:
                cvss = float(v.get("cvss", 0))
            except (TypeError, ValueError):
                continue
            if cvss > 8:
                vid = v.get("id", "")
                desc = v.get("description", "")
                results.append(f"{vid}: {desc}")

    for line in results:
        print(line)

if __name__ == "__main__":
    high_cvss()
```

Задание 3

Для каждого порта — напечатать количество уязвимостей

Описание для тренажера:

1. Прочитать JSON-файл vulnreport.json.

2. Ожидается, что в файле находится список объектов, каждый объект представляет хост и может содержать поле "port" и список уязвимостей в поле "vulns".

3. Для каждого порта посчитать количество уязвимостей на всех хостах с этим портом.

4. Игнорировать объекты без поля "port".

5. Вывести на экран количество уязвимостей для каждого порта в формате "порт: количество", по одной строке на порт.

6. Порты выводить в порядке возрастания.

7. Если файл vulnreport.json отсутствует, вывести сообщение: Файл vulnreport.json не найден.

Тренажер - 20403-print-the-number-of-vulnerabilities

Решение:

```
import json

from collections import Counter



def count_by_port(input_file="vulnreport.json"):

    try:

        with open(input_file, "r", encoding="utf-8") as f:

            hosts = json.load(f)

    except FileNotFoundError:

        print(f"Файл {input_file} не найден")

        return
```

```
    counts = Counter()
    for host in hosts:
        port = host.get("port")
        if port is None:
            continue
        vulns = host.get("vulns", [])
        counts[port] += len(vulns)

    for port in sorted(counts.keys()):
        print(f"{port}: {counts[port]}")


if __name__ == "__main__":
    count_by_port()
```

Задание 4

Найти все уязвимости с CVSS > 8

Преобразовать данные (например, IP, CVE, CVSS) в CSV-таблицу

Сохранить как report.csv

Описание для тренажера:

1. Прочитать CSV-файл vulnerabilities.csv.

2. Ожидается, что в файле есть колонки: IP, CVE, CVSS.

3. Для каждой строки проверить значение CVSS.

4. Отобрать только те строки, у которых CVSS больше 8.

5. Сохранить отобранные данные в новый CSV-файл report.csv с теми же колонками: IP, CVE, CVSS.

6. В начале файла записать заголовки колонок.

7. После успешного сохранения вывести сообщение: Отчет сохранен в report.csv.

8. Если файл vulnerabilities.csv отсутствует, вывести сообщение: Файл vulnerabilities.csv не найден.

Тренажер - 20404-find-all-vulnerabilities

Пример корректного решения:

```
import csv
def filter_high_cvss(input_file="vulnerabilities.csv", output_file="report.csv"):
    Фильтрует уязвимости с CVSS > 8 и сохраняет в CSV
    try:
        with open(input_file, 'r') as infile, open(output_file, 'w', newline='') as outfile:
            reader = csv.DictReader(infile)
            fieldnames = ['IP', 'CVE', 'CVSS']
            writer = csv.DictWriter(outfile, fieldnames=fieldnames)
            writer.writeheader()

            for row in reader:
                try:
                    cvss = float(row['CVSS'])
```

```
                if cvss > 8:
                    writer.writerow(row)
            except (ValueError, KeyError):
                continue

        print(f"Отчет сохранен в {output_file}")
    except FileNotFoundError:
        print(f"Файл {input_file} не найден")
# Пример использования
filter_high_cvss()
```

Критерии оценивания

Критерий	**Баллы**	**Требования**
1. Корректность работы программы	0-1	– 1 балл: программа работает без ошибок, полностью соответствует условиям задачи, и выводит правильный ответ. – 0 баллов: программа не работает или выдает критичные ошибки, выводит неправильный ответ.
2. Использование необходимых функций	0-2	– 2 балла: при решении использованы все требуемые необходимые языковые средства – 1 балл: при решении использованы не все необходимые языковые средства – 0 баллов: требуемые языковые средства не использованы

Шкала оценивания:

Зачтено — 3 балла

Не зачтено — 0 – 2 балла

Требования к оформлению результата выполнения задания: решение задачи предусматривает ввод программного кода в специальное поле тренажера.